
A Branch and Bound Procedure for the Resource Constrained Project Scheduling Problem with Discounted Cash Flows

Author(s): Oya Icmeli and S. Selcuk Erenguc

Source: *Management Science*, Oct., 1996, Vol. 42, No. 10 (Oct., 1996), pp. 1395-1408

Published by: INFORMS

Stable URL: <https://www.jstor.org/stable/2634373>

REFERENCES

Linked references are available on JSTOR for this article:

https://www.jstor.org/stable/2634373?seq=1&cid=pdf-reference#references_tab_contents

You may need to log in to JSTOR to access the linked references.

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Management Science*

JSTOR

A Branch and Bound Procedure for the Resource Constrained Project Scheduling Problem with Discounted Cash Flows

Oya Icmeli • S. Selcuk Erenguc

Cleveland State University, Operations Management Department, Cleveland, Ohio 44115
University of Florida, Decision and Information Sciences Department, Gainesville, Florida 32611

Management of projects is complicated by the scarcity of resources required to execute them. Limited resources usually extend the project completion times beyond those determined by CPM/PERT. Several solution procedures have been developed for solving the resource constrained project scheduling problem. One objective commonly used for these problems is to complete the project as early as possible (minimize makespan). The problem considered in this paper is a resource constrained project scheduling problem, with the added features that there are cash flows associated with the project activities, and the objective is to schedule the project activities in such a way that the net present value of cash flows is maximized. With these features the problem becomes financially motivated and more realistic.

We introduce a branch and bound procedure to solve the resource constrained project scheduling problem with discounted cash flows. Our procedure exploits the known fact that potential resource violations can be eliminated by introducing additional precedence relations between certain project activities. Specifically, we use the “minimal delaying alternatives” concept to resolve resource conflicts. The bounds in the branch and bound procedure are computed by solving Payment Scheduling Problems, which can be formulated as linear programs and by that are well-solvable.

We test our procedure computationally on a set of 90 test problems from the literature and compare it with the only other exact procedure we know of.
(*Project Scheduling; Resource Constraints; Net Present Value*)

1. Introduction

In the last two decades, most of the work in the project scheduling literature has focused on developing solution techniques for project scheduling problems with limited resources (Davis and Heidorn 1971, Davis 1973, Davis and Patterson 1975, Bartusch et al. 1988, and Icmeli et al. 1993). A common objective for these problems is to complete the project as early as possible (minimize makespan). Recently, exact solution procedures for this problem have been proposed by Bell and Park (1990), Demeulemeester and Herroelen (1992). The branch and bound algorithm of Demeulemeester and

Herroelen (1992) introduced the concept of “minimal delaying alternatives” to resolve resource conflicts. In their procedure, each node of the branch and bound tree represents a resource and precedence feasible partial schedule, and resource conflicts are resolved by additional precedence constraints. Bell and Park (1990) introduced an exhaustive search algorithm, A^* search, where resource conflicts are also resolved by imposing additional precedence constraints. However, unlike the branch and bound algorithm of Demeulemeester and Herroelen (1992), corresponding to each node of the search tree is a partially resource feasible complete

schedule. This schedule is obtained by minimizing the makespan of the project scheduling problem in which the resource constraints are relaxed. Recently, Kolisch et al. (1992) studied the effects of problem characteristics on the quality of the solutions obtained by the branch and bound algorithm of Demeulemeester and Herroelen (1992). They generated resource constrained project scheduling problems using their program called PROGEN and concluded that the classical benchmark problems commonly used in the literature do not adequately test the efficacy of exact solution procedures.

In recent years, scheduling models with monetary objectives have received increasing attention (Icmeli et al. 1993). A project scheduling problem with a monetary objective function was first introduced by Russell (1970) as the Payment Scheduling Problem (PSP). The PSP involves identifying a schedule that maximizes the Net Present Value (NPV) of all transactions that occur during the project life, subject to precedence constraints. Grinold (1972) transformed the problem into a Linear Program and suggested two exact solution procedures. Recently, Elmaghraby and Herroelen (1990) presented another exact solution procedure.

Subsequently, the NPV objective was introduced into the resource constrained project scheduling problem with the intention of making it a more realistic model. This problem is recognized as the Resource Constrained Project Scheduling Problem with Discounted Cash Flows (RCPSPDC). The RCPSPDC involves scheduling project activities, with cash inflows and outflows, in such a way that the precedence and resource restrictions are satisfied and the net present value of cash flows is maximized. While many of the recent studies concentrated on the performances of heuristic rules, only a few exact approaches for the solution of this problem have been suggested (see for example, Russell 1986, Smith-Daniels and Aquilano 1987, Padman and Smith-Daniels 1993, Baroum and Patterson 1993, and Icmeli and Erenguc 1994). Early efforts to solve the problem optimally were made by Doersch and Patterson (1977) and by Bey et al. (1981). Doersch and Patterson formulated the problem as a 0–1 Integer Linear Program (ILP) with capital as the only constrained resource. Using a general purpose 0–1 ILP code, small size problems were solved optimally but excessive computation times were required for problems of moderate sizes. The most recent

study on this problem is that of Yang et al. (1992). They present an integer programming algorithm where the solution procedure is a modification of the one that was proposed by Talbot and Patterson (1978). Their computational results indicate that the efficiency of their algorithm is highly dependent on the tightness of the resource constraints and on the project deadline.

The branch and bound procedure we present in this paper solves the RCPSPDC optimally and it does not appear to be nearly as sensitive to the project deadline as the algorithm of Yang et al. (1992). We demonstrate this result through computational experiments that are more comprehensive than those of Yang et al. (1992).

The remainder of this paper is organized as follows. In §2, a conceptual formulation of the RCPSPDC is given. In §3, the search process is explained, and a formal statement of the branch and bound algorithm is given. In §4, a numerical example is presented to illustrate the key points of the algorithm. In §5, computational results are discussed. Finally, our concluding remarks are presented in §6.

2. The Model

The RCPSPDC problem can be written as follows.

$$(P) \quad \text{Max } z(P) = \sum_{i=1}^N q_i e^{-\alpha t_i}, \quad (1)$$

subject to

$$\sum_{S_u} r_{ik} \leq R_{uk} \quad \forall k, u, \quad (2)$$

$$t_j - t_i \geq d_j, \quad (i, j) \in H, \quad (3)$$

$$t_N \leq DD, \quad \text{where} \quad (4)$$

t_i : An integer decision variable specifying the completion time of activity i and t_N denotes the completion time of the project.

Parameters

N : The activity which is a successor of all other activities in the project.

DD : The latest time the project must be completed.

q_i : The compounded value of cash flows that occur during the implementation of activity i at its completion time.

$$q_i = \sum_{L=1}^{d_i} f_{iL} e^{\alpha(d_i-L)}, \quad i = 1, \dots, N. \quad (5)$$

d_i : Duration of activity i .

f_{iL} : Cash flow for activity i in period L of its duration, $L = 1, \dots, d_i$, f_{iL} may be negative, zero or positive.

α : Discount rate.

r_{ik} : Per period usage of resource k by activity i .

R_{uk} : Availability level of resource k , $k = 1, \dots, K$, at time period u .

H : Set of precedence relationships among the activities; $H = \{(i, j) | i \text{ precedes } j\}$.

S_u : Set of activities that are active at time period u . Note that S_u is a function of the choice of t_i .

The first set of constraints ensures that resources allocated to activities at any time during the project duration do not exceed resource availabilities. The second set of constraints enforces the precedence relations between the activities. Specifically, an activity can start only after all the activities which must technologically precede that activity are completed. The last constraint ensures that the project ends by the latest allowable completion time. We note that following the RCPSP literature problem P is stated in the Activity-on-Node (AON) format.

The objective is to maximize the discounted value of all cash flows that occur during the life of the project. Note that early schedules do not necessarily maximize the NPV of cash flows. Depending on the f_{iL} values associated with an activity, it may be profitable to delay that activity. The latest project completion time, DD , is determined by using the approach given in Icmeli and Erenguc (1994).

In the model, we assume that each activity: (1) has a known, deterministic, integer duration; (2) uses a known, deterministic and constant level (might be zero) of integral resources of each kind during its processing time; and (3) cannot be preempted.

3. A Branch and Bound Procedure for RCPSPDC

Schedules for the resource constrained problem that are optimal with respect to the makespan objective do not necessarily maximize the NPV of cash flows (Russell 1970). As indicated in Grinold (1972), if a project man-

ager has to make a large enough lump sum payment at the end of the project, an optimal policy would be not to complete the project at all. Thus, typical procedures that find the earliest completion time for a project will not in general provide a good solution when cash flows are considered. Therefore, new solution procedures need to be developed to manage effectively the cash flows and resource usage in projects.

In this section, a branch and bound procedure is developed for the RCPSPDC. Although there are several exact solution techniques for the resource constrained problem with the makespan minimization objective (Demeulemeester and Herroelen 1992 and Christofides et al. 1987), to our knowledge research on exact solution techniques for RCPSPDC is relatively recent and sparse. We are motivated by the opportunity for providing a set of benchmark problems with their optimal solutions which can be used for testing the quality of exact and heuristic solution procedures to be developed in the future.

The following is used for the development of the branch and bound procedure for RCPSPDC. Let P' be the problem obtained by removing the resource constraints from the constraint set of problem (P) .

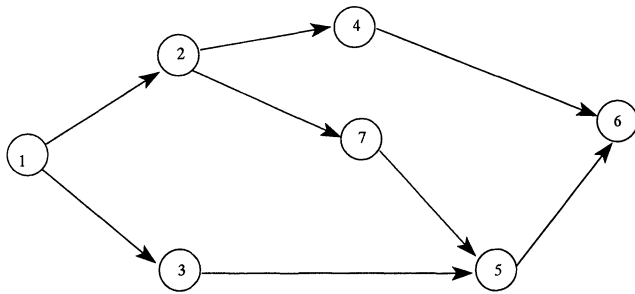
PROPOSITION. P' and Payment Scheduling Problem (PSP) are equivalent problems.

PROOF. See the appendix.

In his paper, Grinold (1972) has shown that the PSP is equivalent to a linear program and that the problem has an optimal solution corresponding to a spanning tree. Grinold developed two algorithms, the first of which is the Fixed Deadline Algorithm (FDA) that solves the problem for a given deadline. The second algorithm finds optimal solutions for all possible deadlines. The Fixed Deadline Algorithm is used in our branch and bound procedure to solve the subproblem at each node of the branch and bound tree optimally.

Our bounding scheme bears a resemblance to that of Bell and Park (1990) in that at each node of our branch and bound tree, a complete schedule, which may be resource infeasible, is obtained. It should be noted that whereas the subproblem solved in our procedure to produce the above-mentioned complete schedules are PSPs, in the A^* search these schedules are determined by implementing CPM.

Figure 1 Activity on Node Representation of the Project Network



In our branch and bound procedure, branching is done using the “minimal delaying alternatives” concept of Demeulemeester and Herroelen (1992). In the next section, we introduce our search process. The notation we use was adapted from the same paper.

3.1. The Search Process

At the initial node of the tree an optimal solution to problem P' is obtained by using the FDA. The objective function value, UB^0 , is an upper bound for problem (P) . If this solution is resource feasible, then it is also an optimal solution for (P) , and the procedure terminates at this level. If, however, there is at least one period with at least one resource conflict, then we branch from this solution to generate the next level of the branch and bound tree. At this level, several new branches are generated to resolve the resource conflict in question. The resource conflicts are resolved by appending additional precedence relations to P' . The subproblems thus obtained are solved to optimality as PSPs using the FDA. Each newly generated branch leads into a node in the branch and bound tree, and associated with each node, there is a set of additional precedence relations which are imposed on problem P' . Upon solving the corresponding PSP, a node is fathomed if either the optimum PSP solution has a project duration that exceeds DD , or the solution to PSP is resource feasible, or the optimum PSP objective value is less than or equal to that of the best feasible solution found so far, incumbent solution. If a node can not be fathomed, it is added to the candidate list, NB . Among the nodes in NB , the node with the greatest objective function value is selected for further branching and it is removed from the list. The algorithm terminates when NB becomes empty.

Let $P'(k)$ be the subproblem at node k . $P'(k)$ includes all the additional precedence relations that were imposed on the path from the initial node to node k in the branch and bound tree. Note that $P'(0) = P'$. Furthermore, let C^k be the set that contains all the activities which are active at the earliest time period where a resource conflict occurs in the schedule obtained by solving $P'(k)$. The delaying set D^k consists of all subsets of activities D_q^k , $q = 1, \dots, Q$, the delay of which would resolve the particular conflict within the activities in C^k . The delaying alternative D_q^k is minimal if it does not contain other delaying alternatives as a subset (Demeulemeester and Herroelen 1992). Each delaying alternative D_q^k becomes eligible to be scheduled at the completion time of some activity in $C^k \setminus D_q^k$. The delay of D_q^k is introduced by defining additional precedence relations between the activities of D_q^k and $C^k \setminus D_q^k$.

In order to resolve a resource conflict, it is sufficient to consider only minimal delaying alternatives (Demeulemeester and Herroelen 1992). To illustrate the construction of delaying sets we give the following example. Assume that there is a resource conflict at time period t when activities 2, 3, and 4 are scheduled to occur simultaneously at node k . Then, $C^k = \{2, 3, 4\}$. Let us assume further, without loss of generality, that delaying 2 and 3 together or delaying 4 alone would resolve the conflict at that time period t . Thus, $D^k = \{\{2, 3\}, \{4\}\}$. We have two delaying alternatives: $D_1^k = \{2, 3\}$ and $D_2^k = \{4\}$. The delay of D_1^k is introduced by defining two additional precedence relations between activities 4 and 2 and between 4 and 3. Similarly, the delay of D_2^k is introduced by defining two precedence relations between the activities 2 and 4 and between 3 and 4. Note that the total number of branches, and therefore nodes, generated is 3: one branch corresponds to delaying activities 2 and 3 together after activity 4. The other two correspond to delaying activity 4 after activity 2 and delaying 4 after activity 3.

In order to facilitate the presentation of the branch and bound algorithm we introduce the following notation:

p : The level of the branch and bound tree.

B^k : Total number of branches generated at node k of the tree.

H^k : The set of precedence relations that the network has at the k th node of the tree. These include the original

precedence restrictions and those that have been appended so far.

LB : The objective function value of the incumbent solution, x_{INC} , i.e., a lower bound for the optimal value of (P) . Initially, LB is set equal to the value of a feasible solution if one is available, otherwise it is set equal to $-\infty$.

NB : The candidate list, i.e., the set of nodes which are not yet branched from.

x_k : The solution obtained by solving $P'(k)$.

x_{INC} : The incumbent solution, i.e., the best solution found so far for problem (P) .

UB^k : The optimal objective function value of the subproblem at node k .

PR_k^b : The set of precedence relations imposed on problem $P'(k)$ at branch b , emanating from node k to resolve the resource conflict in question. Note that at the newly generated node that corresponds to branch b , the total set of precedence relationships becomes $(H^k \cup PR_k^b)$.

Θ_p : Set of all unfathomed nodes of level p .

δ_k : The length of the minimum spanning tree of the schedule obtained at node k .

3.2. The Branch and Bound Algorithm

Step 0. Let $p = 0, k = 0, LB = -\infty, NB = \emptyset, \Theta_p = \emptyset$. Determine a tolerance level, ϵ . Solve problem $P'(0)$: $\max f(x)$ s.t. $x \in H^0$. If the optimal solution, x_0 , is resource feasible, then, stop and return UB^0 and x_0 as the optimal solution to problem (P) ; else continue; Add node 0 to the candidate list NB . Update Θ_p .

Step 1. Remove node k from NB . Determine the first resource conflict of the schedule obtained at node k .

- 1.1. If no resource conflict occurs, then go to Step 3. Else, determine C^k and D^k . Branch from node k to resolve this conflict. That is, determine B^k and $PR_k^b, b \in \{1, 2, \dots, B^k\}$. Let $p \leftarrow (p + 1)$.

Step 2. Branch from node k .

- 2.1. Corresponding to each branch $b = 1, 2, \dots, B^k$ from node k , generate a new node $k + b$ and solve $P'(k + b)$: $\max f(x)$ s.t. $x \in H^{k+b}$. If either $\delta_{k+b} > DD$ or $UB^{k+b} \leq LB(1 + \epsilon)$, then fathom node $k + b$, else add node $k + b$ to NB and update Θ_p . If no node is added to NB , then go to Step 3, else continue.
- 2.2. Choose node index n which satisfies,
 $UB^n = \max\{UB^i: i \in \Theta_p\}$.
 $k \leftarrow n$.

- 2.3. If x_k is resource feasible, then, $LB \leftarrow UB^k, x_{INC} = x_k$. Fathom all the nodes in Θ_p , set $\Theta_p = \emptyset$ and delete them from NB . Go to Step 3.
 Else go to Step 1.

Step 3. Backtrack.

- 3.1. Set $p \leftarrow (p - 1)$. If $p = 0$, then, go to Step 4. Else continue.
- 3.2. Fathom each node $n \in \Theta_p$, if $UB^n \leq LB(1 + \epsilon)$ and delete it from NB . Choose node index n which satisfies,
 $UB^n = \max\{UB^i: i \in \Theta_p\}$.
 $k \leftarrow n$.
- 3.3. If there exists no unfathomed node at this level ($\Theta_p = \emptyset$), go to Step 3.1.
 Else go to Step 1.

Step 4. Termination. Return LB and x_{INC} as the optimal solution to problem (P) .

Note that the initial LB for the problem can be obtained by a heuristic procedure.

4. An Example

In this section we give an example of the branch and bound procedure for RCPSPDC. Initially there are seven activities and eight precedence relationships (Figure 1). The deadline of the project is set to 12 and the discount factor is 0.01 per period. The following table summarizes the data for the problem.

Assume that we have one resource type with availability level of 3. q_i, d_i , and r_{i1} are the cash flow, the duration, and the resource usage of activity i , respectively.

At $p = 0$, the value of the optimum solution to the node zero subproblem, $P'(0)$ is $UB^0 = 845.71$ where LB

Table 1

Activity	q_i	d_i	r_i	Immed. Successor
1	0	1	2	2, 3
2	200	2	2	4, 7
3	-300	2	2	5
4	400	3	1	6
5	700	2	1	6
6	0	1	2	-
7	-100	1	1	5

$= -\infty$. The corresponding schedule is given in Table 2. Since this solution is not resource feasible, we generate the next level of the tree, $p = 1$, and we determine the first resource conflict, which occurs at time period 3: $C^0 = \{2, 3\}$ and $D^0 = \{\{2\}, \{3\}\}$. Thus, we have two branches emanating from node 0 to resolve the resource conflict: 3 precedes 2 at node 1 of the branch and bound tree and 2 precedes 3 at node 2 (Figure 2). Two additional nodes are generated. At the first one we solve the subproblem with the additional precedence relation between 3 and 2, while at the second node we solve the problem with the additional precedence relation between 2 and 3. In the first node we obtain $UB^1 = 820.36$, and in the second $UB^2 = 842.97$. Since $UB^2 > UB^1$, we proceed from node 2, node 1 is added to the candidate list NB (Figure 3). The solution obtained at node 2 is not resource feasible either. Resource conflict occurs at $t = 5$ with $C^1 = \{3, 4, 7\}$ and $D^1 = \{\{3\}, \{4\}, \{7\}\}$. The procedure moves to level 2, i.e., $p = 2$. The total number of branches generated from node 2 to resolve the resource conflict is 6. At each node we solve a subproblem with the additional precedence relation between the activities 3, 4 and 7 as indicated in Figure 2. The solution found at node 4 has the highest objective function value and is resource feasible (Figure 4). Thus, we update the current lower bound, $LB = 838.36$. This causes nodes 3, 5, 6, 7 and 8 to be fathomed. The procedure then backtracks to node 1 at level 1, and fathoms this node since $UB^1 < LB$. Subsequently NB becomes empty and the algorithm terminates. The solution found at node 4 is the optimal solution to RCPSPPDC.

Table 2

Activity	Nodes								
	0	1	2	3	4*	5	6	7	8
1	1**	1	1	1	1	1	1	1	1
2	3	5	3	3	3	3	3	3	3
3	4	3	5	8	6	5	5	5	7
4	6	8	6	6	6	8	7	6	6
5	6	8	7	10	8	7	7	8	9
6	7	9	8	11	9	9	8	9	10
7	4	6	5	8	4	5	4	6	7

**: Completion time of each activity

*: Optimal schedule

5. Computational Results

In order to test the efficiency of the branch and bound algorithm, a set of 90 test problems with three different parameter settings was solved. The algorithm was programmed in FORTRAN and run on an IBM3090 computer with vector processor. Fifty of the test problems came from a set that was assembled by Patterson (1984). These problems have been commonly used as benchmarks for project scheduling algorithms and contain up to three resource types. We grouped the test problems into six sets according to the number of activities which range from 7 to 51. There are eight problems with less than 22 activities in the first set, 23 problems with 22 activities and one with 23 activities in the second set, 11 with 27 activities in the third set, two with 35 activities in the fourth set, and finally five with 51 activities in the fifth set. Only seven problems have less than three resource types (five in the first set and two in the fourth). In Demeulemeester and Herroelen (1991), seven problems were identified as difficult to solve due to tight resource constraints. We used six of these seven problems in our computational experiments. Since the activity cash flows, q_i s, were not available in the data set, we generated them randomly from a uniform distribution on $[-500, 1000]$. In all of the 90 problems, resource availabilities were constant over time.

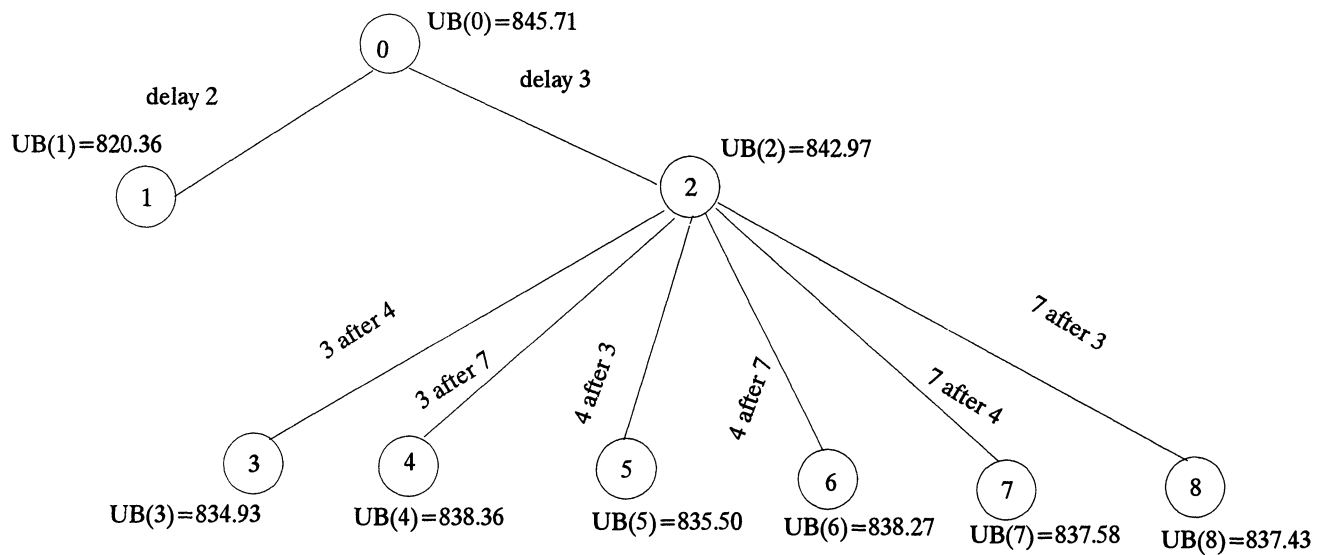
The sixth set consists of forty problems which we generated using PROGEN. Each problem in this set has thirty two activities and three resource types. For these problems cash flows were generated randomly from a uniform distribution on $[-5000, 10,000]$.

Table 3 summarizes problem and network characteristics for each set as defined in Kolisch et al. (1992). In column three the average network complexity, C , of each set is given. It is calculated as the ratio of the total number of precedence relationships to the total number of activities in the network. Average Resource Strength (RS_r) for each resource type r is computed as follows and is given in column 4.

$$RS_r = \frac{K_r - K_r^{\min}}{(K_r^{\max} - K_r^{\min})}. \quad (6)$$

Here K_r is the resource availability per time period, $K_r^{\min}$ the minimal demand, and $K_r^{\max}$ the maximum possible demand for each resource type r . Note that $0 \leq RS_r \leq 1$ and as RS_r approaches to 1, resource r becomes less

Figure 2 The Branch and Bound Tree



constraining. Average Utilization Factor (UF_r) for each resource type r is computed as follows and given in column five.

$$UF_r = \frac{K_r(\text{due date})}{\sum_j (k_{jr}d_j)} \quad (7)$$

Here for each resource type r and activity j , k_{jr} is the resource usage per time period and d_j is the duration.

Note that as UF_r values get larger, resources would become less constraining. The values reported in Table 3 indicate that our test problems are of at least moderate difficulty (Kolisch et al. 1992).

In the implementation of the algorithm, limits of 600 seconds and 4000 subproblems were imposed on the CPU time and the number of subproblems solved, respectively. For each test problem the objective function

value of the feasible solution generated by the heuristic procedure of Icmeli and Erenguc (1994) was used as the initial lower bound.

At the initial stage, the algorithm was tested with 0% tolerance and 0.01 discount rate. Table 4 gives a summary of these computational results. The second column in the table gives the number of problems successfully solved. Of the 90 test problems attempted, 34 were solved to optimality within the given limits on the number of subproblems and CPU time. For 16 problems with 22 activities, three problems with 27 activities, and seven problems with 32 activities, the algorithm terminated prematurely by reaching the 4000 subproblem limit (column 3). Column 4 gives the number of problems terminated prematurely as a result of reaching the 600 second CPU time limit. In all the premature

Figure 3 The Project Network at Node 2

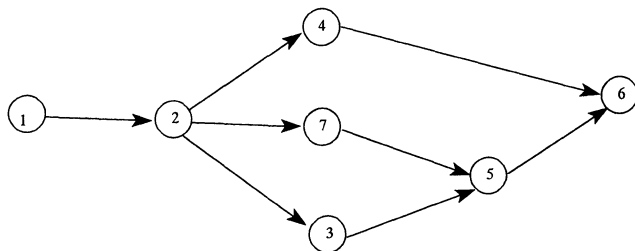


Figure 4 The Project Network at Node 4

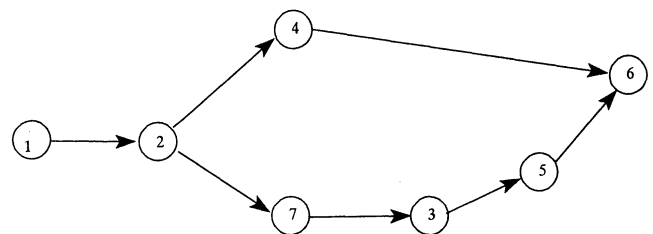


Table 3 Problem Characteristics

No. of Activities	No. of Problems	Average Network Complexity (C)	Average RS _r			Average UF _r		
<22	8	1.30	0.12	0.50	0.68	3.67	5.20	5.36
22, 23	24	1.61	0.08	0.12	0.12	2.81	2.80	3.06
27	11	1.37	0.11	0.10	0.10	2.23	2.06	2.11
35	2	1.65	0.10	0.53	—	1.75	2.59	—
51	5	1.52	0.07	0.06	0.06	2.11	2.07	1.77
32	40	1.80	0.55	0.55	0.55	3.88	3.85	3.88

termination cases, the best solution found was recorded before termination.

For the problems that were successfully solved, the average and the range of the number of subproblems solved are given in column 5. The first entry in each cell corresponds to the average value and the second entry corresponds to the range of values obtained for each set. Column 6 shows the average and the range of the number of simplex (Grinold 1972) pivots performed. In its implementation, rather than solving a new subproblem from scratch, the branch and bound procedure updates the solution obtained at the parent node of a newly generated node and uses it as a starting solution for re-optimization at the new node. Column 7 shows the average and range of the number of branches that were generated at a node. Column 8 gives the range of the number of precedence relations added at the optimal solution. Column 9 gives the range of the candidate list size. Finally, the last column gives the average and the range of CPU seconds required.

We then attempted to solve those problems for which the procedure terminated prematurely, (columns 3 and 4 of Table 4), with larger tolerance factors. Tables 5–7 summarize these results. Sixteen of the 56 problems were solved to optimality with the tolerance set at 0.005. Of the remaining 40 problems, the algorithm reached the subproblem limit for 15 and the time limit for 25 and therefore terminated prematurely. When the tolerance level was increased to 0.025, 20 of the remaining 40 problems were solved successfully. For only 11 of the remaining 20 problems, the algorithm came to a normal termination when the tolerance was increased to 0.05. The nine problems that remained unsolved were all generated by PROGEN. For the problems that were suc-

cessfully solved, the average number of nodes generated is significantly lower than the maximum limit of 4,000. Furthermore, the number of precedence relations added to obtain an optimal solution and the size of the branch and bound tree (the depth and the width of the tree) did not increase significantly with the problem size (number of activities). Moreover, a noticeable increase in the CPU times were observed with increasing problem size and/or resource tightness. We also note that, significant improvements in lower bounds occurred early in the procedure and subsequent improvements were in general marginal.

In order to analyze the effect of discount rate on the computational effort, additional tests were done with larger discount rates. Tables 8 and 9 summarize the computational results with discount factors of 0.05 and 0.10, respectively. In this part of the computational study only those problems that were solved to optimality in Table 4 were considered. The results indicate that the discount factor did not have a significant impact on the performance of the algorithm.

Finally, a comparative testing was done with the only existing exact procedure in the literature. The only other exact algorithm for solving RCPSPDC has been reported by Yang, Talbot, and Patterson (YTP) (1992). The authors suggested a depth-first branch and bound algorithm which employs two fathoming criteria based on the network cut concept. They tested their algorithm on 10 problems taken from the Patterson data set. As indicated in their paper, the algorithm is very sensitive to project deadline. Computational requirements increase dramatically with the length of the project deadline. They report computational experience in which the latest project completion time DD is set equal to $D + k$,

Table 4 B&B Results with 0% Tolerance and 0.01 Discount Rate

No. of Activities (1)	No. of Problems Successfully Solved (2)	No. of Problems Terminated by Reaching 4000 Subproblem Limit (3)	No. of Problems Terminated by Reaching 600 sec. CPU Time Limit (4)	Average and Range of the No. of Subproblems Solved (5)	Average and Range of the No. of Pivots Per Problem (6)	Average and Range of No. of Branches Generated at a Node (7)	Range of No. of Precedence Relations Added at Optimal Soln. (8)	Range of the List Size (9)	Average and Range of CPU Time Consumed Per Problem (sec.) (10)
<22	8	-	-	366.87 [1, 1798]	1481.50 [4, 5830]	4.45 [0, 24]	[0, 5]	[1, 62]	18.22 [0.01, 98.08]
22	6	16	1	313.16 [1, 1011]	1866.50 [9, 6565]	3.96 [0, 12]	[0, 5]	[1, 27]	21.95 [0.25, 94.59]
23	-	-	1	-	-	-	-	-	-
27	-	3	8	-	-	-	-	-	-
35	2	-	-	266.50 [99, 453]	1891.00 [944, 2838]	8.15 [0, 20]	[2, 3]	[35]	208.41 [103.82, 313.00]
51	1	-	4	7.00 [7]	51 [51]	6.00 [6]	[1]	[4]	17.19 [17.19]
32	17	7	16	40.43 [1, 150]	393.53 [2, 3207]	7.32 [0, 24]	[0, 4]	[1, 51]	25.74 [0.20, 209.92]

Table 5 B&B Results with 0.5% Tolerance and 0.01 Discount Rate

No. of Activities (1)	No. of Problems Successfully Solved (2)	No. of Problems Terminated by Reaching 4000 Subproblem Limit (3)	No. of Problems Terminated by Reaching 600 sec. CPU Time Limit (4)	Average and Range of the No. of Subproblems Solved (5)	Average and Range of the No. of Pivots Per Problem (6)	Average and Range of the No. of Branches Generated at a Node (7)	Range of Precedence Relations Added at Optimal Soln. (8)	Range of the List Size (9)	Average and Range of CPU Time Consumed Per Problem (sec.) (10)
22	9	8	-	2069.12 [385, 2883]	7718.12 [1449, 13949]	5.04 [8, 19]	[7, 15]	[26, 84]	196.69 [41.48, 369.88]
23	1	-	-	92.00 [92]	430.00 [430]	5.75 [0, 9]	[18]	[76]	22.36 [22.36]
27	2	1	8	304.00 [67, 551]	1465.50 [125, 2806]	7.04 [0, 17]	[12, 19]	[57, 149]	79.10 [10.55, 147.66]
51	-	-	4	-	-	-	-	-	-
32	4	6	13	592.66 [454, 1233]	3534 [821, 5779]	10.70 [16, 57]	[3, 4]	[10, 262]	234.20 [54.53, 382.91]

Table 6 B&B Results with 2.5% Tolerance and 0.01 Discount Rate

No. of Activities (1)	No. of Problems Successfully Solved (2)	No. of Problems Terminated by Reaching 4000 Subproblem Limit (3)	No. of Problems Terminated by Reaching 600 sec. CPU Limit (4)	Average and the Range of the No. of Subproblems Solved (5)	Average and the Range of the No. of Pivots Per Problem (6)	Average and the Range of the No. of Branches Generated at a Node (7)	Range of No. of Precedence Relations Added at Optimal Soln. (8)	Range of the List Size (9)	Average and Range of CPU Time Consumed Per Problem (sec.) (10)
22	6	2	—	215.50 [82, 421]	778.66 [196, 1401]	5.79 [0, 20]	[5, 22]	[7, 77]	22.26 [5.63, 39.95]
27	7	—	2	348.85 [105, 1176]	1754.57 [462, 6321]	7.28 [0, 20]	[13, 21]	[10, 109]	90.09 [23.76, 323.93]
51	—	—	4	—	—	—	—	—	—
32	7	3	9	500.40 [94, 892]	3517.20 [211, 8671]	21.10 [10, 48]	[12, 49]	[81, 858]	316.96 [23.65, 579.30]

Table 7 B&B Results with 5% Tolerance and 0.01 Discount Rate

No. of Activities (1)	No. of Problems Successfully Solved (2)	No. of Problems Terminated by Reaching 4000 Subproblem Limit (3)	No. of Problems Terminated by Reaching 600 sec. CPU Limit (4)	Average and the Range of the No. of Subproblems Solved (5)	Average and the Range of the No. of Pivots Per Problem (6)	Average and the Range of the No. of Branches Generated at a Node (7)	Range of No. of Precedence Relations Added at Optimal Soln. (8)	Range of the List Size (9)	Average and Range of CPU Time Consumed Per Problem (sec.) (10)
22	2	—	—	156.00 [113, 199]	460.50 [406, 521]	5.27 [0, 20]	[15, 20]	[37, 75]	13.20 [11.35, 15.05]
27	2	—	—	301.00 [13, 581]	1453.00 [46, 2860]	9.70 [0, 12]	[1, 3]	[2, 15]	73.97 [2.34, 145.61]
51	4	—	—	34.50 [7, 105]	142.00 [54, 361]	9.75 [0, 12]	[0, 18]	[2, 91]	48.03 [18.38, 122.48]
32	3	3	6	218.66 [98, 329]	2054.33 [1037, 3634]	9.69 [6, 35]	[2, 3]	[52, 251]	138.06 [69.92, 244.71]

Table 8 B&B Results with 0% Tolerance and 0.05 Discount Rate

No. of Activities (1)	No. of Problems Successfully Solved (2)	No. of Problems Terminated by Reaching 4000 Subproblem Limit (3)	No. of Problems Terminated by Reaching 600 sec. CPU Time Limit (4)	Average and Range of the No. of Solved Subproblems (5)	Average and Range of the No. of Pivots Per Problem (6)	Average and Range of No. of Branches Generated at a Node (7)	Range of No. of Precedence Relations Added at Optimal Soln. (8)	Range of the List Size (9)	Average and Range of CPU Time Consumed Per Problem (sec.) (10)
<22	8	-	-	168.87 [1, 1204]	585.50 [4, 4307]	4.68 [0, 18]	[1, 7]	[1, 53]	5.24 [0.01, 38.05]
22	6	-	-	249.33 [1, 744]	1508.00 [9, 5052]	3.94 [0, 12]	[0, 7]	[1, 32]	48.82 [0.26, 146.81]
35	2	-	-	307.50 [113, 502]	2280.50 [1199, 3362]	7.80 [0, 20]	[5, 7]	[35, 66]	256.28 [134.14, 378.42]
51	-	-	1	-	-	-	-	-	-
32	17	-	-	102.10 [1, 557]	134.40 [2, 557]	6.20 [0, 34]	[0, 3]	[1, 44]	8.68 [0.15, 35.23]

Table 9 B&B Results with 0% Tolerance and 0.10 Discount Rate

No. of Activities (1)	No. of Problems Successfully Solved (2)	No. of Problems Terminated by Reaching 4000 Subproblem Limit (3)	No. of Problems Terminated by Reaching 600 Sec. CPU Time Limit (4)	Average and Range of No. of Solved Subproblems (5)	Average and Range of Pivots Per Problem (6)	Average and Range of No. of Branches Generated at a Node (7)	Range of Precedence Relations Added at Optimal Soln. (8)	Range of the List Size (9)	Average and Range of CPU Time Consumed Per Problem (sec.) (10)
<22	8	-	-	95.37 [1, 658]	350.62 [4, 2477]	4.69 [0, 18]	[1, 7]	[1, 53]	3.10 [0.01, 21.68]
22	6	-	-	156.66 [1, 326]	879.16 [9, 2274]	3.92 [0, 12]	[0, 6]	[1, 50]	25.50 [0.26, 66.17]
35	-	-	1	346.00 [346]	4098.00 [4098]	10.48 [0, 20]	[13]	[66]	390.37 [390.37]
51	-	-	1	-	-	-	-	-	-
32	17	-	-	57.60 [1, 350]	86.40 [2, 350]	11.69 [0, 24]	[0, 3]	[1, 32]	3.65 [0.15, 22.03]

Table 10

No. of Activities	No. of Problems Successfully Solved with IE	Average CPU Times (sec.) with IE	No. of Problems Successfully Solved with YTP	Average CPU Time (sec.) with YTP
<22	6	0.20	7	0.09
22	20	53.39	3	19.14
23	1	51.38	—	—
27	7	70.73	—	—
35	1	282.80	—	—
51	1	336.95	—	—
32	19	37.98	—	—

IE: stands for Icmeli and Erenguc's branch and bound algorithm.

YTP: stands for Yang, Talbot, and Patterson's branch and bound algorithm.

— for these problems for YTP algorithm terminated after reaching the CPU limit of 600 seconds.

where D is the minimum resource constrained duration (makespan) and $k = 1, 2, 3$.

In our comparative testing, the discount rate was set equal to 0.01, tolerance to 0, and latest project completion time, DD , to $D + 3$. The initial lower bounds were taken as $-\infty$. These are the parameter settings used in the YTP study. A CPU time limit of 600 seconds was imposed. We attempted to solve the 90 problems, with the updated project deadlines, using both the YTP procedure and our branch and bound procedure. Table 10 summarizes the computational results. Whereas our algorithm provided optimal solutions for 55 of the 90 problems, the YTP algorithm could solve only 10. It exceeded the CPU time limit for the remaining 80 problems. It is instructive to note that the number of problems that our algorithm could solve to exact optimality (zero tolerance) is greater for the new (shorter) deadlines than that for the longer deadlines used in Table 4. We also attempted to solve the 90 problems with the extended deadlines of Table 4 with the YTP procedure. These project deadlines are, on the average, 14 time units longer than the minimum resource constrained durations. The YTP procedure failed to produce a solution for any of the 90 problems within the limit of 600 CPU seconds. (Recall that our algorithm solved 34 of the test problems successfully with 0 tolerance). This is consistent with the observation that the YTP procedure is very sensitive to the length of the project duration as reported in Yang et al. (1992). Another significant ad-

vantage of our procedure is that even when the procedure terminates prematurely, either due to subproblem or CPU time limit, it reports the best feasible solution found so far. In fact for the 90 problems that we solved, our procedure always produced several feasible solutions. Of the six difficult problems mentioned above, one problem was solved to optimality, one was solved within 0.005 of optimality, two problems were solved within 0.025 of optimality, and the last two were solved within 0.05 of optimality within the imposed limits. By its design, the first feasible solution found in the YTP procedure is in fact an optimal solution. Therefore, if the procedure terminates prematurely, one does not have a feasible solution for the problem. Tables 4–10 provide strong support in favor of the relative efficiency of our algorithm.

6. Conclusions

In this paper, we presented a depth-first branch and bound algorithm for the resource constrained project scheduling problem with discounted cash flows. The algorithm finds an optimal solution in a finite number of steps. In implementing the algorithm, we adopted tolerance levels which guarantee that the solution value obtained by the algorithm is within $(100\epsilon)\%$ of that of the optimal solution. The smallest and the largest values we used for ϵ were 0 and 0.05, respectively. In our computational testing we used 50 Patterson and 40 PRO-

GEN problems. Our computational results proved to be rather encouraging, and our algorithm significantly outperformed the only other known procedure for the problem.¹

¹ The authors would like to express their sincere thanks to Professor James H. Patterson for providing his data set and the YTP code, to Professor Walter O. Rom for his valuable comments and to three anonymous referees for their extremely helpful comments and suggestions. They also thank Professor Rainer Kolisch for making the PROGEN software available.

Appendix

Before giving the proof of the proposition we present the formulation of PSP and define the notation that we use in the proof. In the literature, PSP is formulated as follows (Grinold 1972): (PSP)

$$\text{Max } z(\text{PSP}) = \sum_{v=1}^M c_v e^{-\alpha T_v}, \quad (8)$$

subject to

$$T_z - T_v \geq d_j \quad \text{for } z = 2, \dots, M, v \in \mathcal{P}_z, \quad (9)$$

$$T_M \leq DD, \quad (10)$$

$$T_v \geq 0, \quad v = 1, \dots, M, \quad \text{where} \quad (11)$$

T_v : A decision variable specifying the realization time of event v , $v = 1, \dots, M$, $T_1 = 0$.

Parameters:

M : Total number of events in the project.

c_v : Cash flow at the realization time of event v .

d_j : Duration of the activity j whose tail node is v and head node is z .

DD : The latest time the project must be completed.

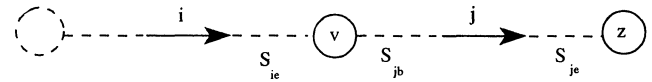
α : Discount rate.

$\mathcal{P}_z$: Set of events immediately preceding event z .

Note that in this project network format each node represents an event signalling the completion of all activities leading into it.

Consider Figure 5. Assume without loss of generality that activity i immediately precedes activity j , and that node v is the head node of activity i , and node z is the head node of activity j , and $v, z \in \{1, \dots, M\}$. Let S_j be the total slack time between the events v and z . Let $S_j = s_{jb} + s_{je}$ where s_{jb} represents the slack time between the realization of event v and the start of activity j . Also let s_{je} be the slack time between the completion time of activity j and the realization time of event z . Similarly, let s_{ie} be the slack time between the completion time of activity i and the realization time of event v . Note that $S_j, s_{jb}, s_{je}, s_{ie} \geq 0$. Unless otherwise stated, the notation used in §2 is adopted here. From the definition of t_i , $i = 1, \dots, N$ and T_v , $v = 1, \dots, M$, it must be clear that

Figure 5



$$t_j \in [T_v + d_j, T_z] \quad \forall j, \quad (12)$$

$$T_z = \max_{j \in G_z} \{t_j\} \quad \forall z, \quad (13)$$

where G_z represents the set of all activities leading into event z .

PROPOSITION. P' and PSP are equivalent problems.

PROOF. The proof consists of three parts.

(i) Given a feasible solution, $T = (T_1, \dots, T_M)$, to PSP, a corresponding feasible solution, $t = (t_1, \dots, t_N)$, to P' is obtained from (12): If T is feasible, then for any pair of events v, z where $v \in \mathcal{P}_z$, $T_z - T_v \geq d_j \Rightarrow T_z - T_v = d_j + S_j$, then it follows from (12) that

$$(t_j + s_{je}) - (t_i + s_{ie}) = d_j + S_j \Rightarrow t_j - (t_i + s_{ie}) = d_j + (S_j - s_{je}) \Rightarrow$$

$$t_j - (t_i + s_{ie}) = d_j + s_{jb} \Rightarrow t_j - t_i \geq d_j + s_{jb} \Rightarrow t_j - t_i \geq d_j.$$

(ii) Given a feasible solution, $t = (t_1, \dots, t_N)$, to P' , a corresponding feasible solution, $T = (T_1, \dots, T_M)$ to PSP is obtained from (13): If t is feasible, then for any pair of activities i, j where $(i, j) \in H$,

$$t_j - t_i \geq d_j \Rightarrow t_j - t_i - (s_{ie} + s_{jb}) = d_j \Rightarrow t_j - (t_i + s_{ie}) - s_{jb}$$

$$= d_j \Rightarrow t_j - T_v - s_{jb} = d_j \Rightarrow t_j - T_v \geq d_j.$$

It follows from (13) that $T_z \geq t_j$, then $T_z - T_v \geq d_j$.

Furthermore, by the definition of N , $t_N = T_M$. Thus, $T_M \leq DD \Leftrightarrow t_N \leq DD$.

(iii) At optimality, P' and PSP have the same objective value: It has been noted that (Sepil 1987), at an optimal solution to PSP, an activity L with nonnegative cash flow ($q_L \geq 0$) will be scheduled to start at its tail node and an activity w with a negative cash flow ($q_w < 0$) will be scheduled to complete at its head node and as a result, the following relationship between the coefficients of the two objective functions holds:

$$c_v = \sum_{L \in F_v} q_L e^{-\alpha d_L} + \sum_{w \in P_v} q_w, \quad \text{where} \quad (14)$$

$q_L \geq 0$, $q_w < 0$ and F_v is defined as the set of activities with nonnegative cash flows that emanate from event v and P_v represents the set of all activities with negative cash flows that lead into event v .

Then, we can substitute c_v as follows:

$$z(\text{PSP}) = \sum_{v=1}^M c_v e^{-\alpha T_v} \Rightarrow$$

$$z(\text{PSP}) = \sum_{v=1}^M \left[\sum_{L \in F_v} q_L e^{-\alpha d_L} + \sum_{w \in P_v} q_w \right] e^{-\alpha T_v} \Rightarrow$$

$$z(\text{PSP}) = \sum_{v=1}^M \left[\sum_{L \in F_v} (q_L e^{-\alpha d_L}) e^{-\alpha T_v} + \sum_{w \in P_v} q_w e^{-\alpha T_v} \right].$$

By (13), for each node v ,

$$T_v = \max_{w \in G_v} \{t_w\} = t_{w'(v)}.$$

But since for each $w \in P_v$, $q_w < 0$, then, $t_{w'(v)} = t_w$.

Thus,

$$z(\text{PSP}) = \sum_{v=1}^M \left[\sum_{L \in F_v} q_L e^{-\alpha(d_L + t_w)} + \sum_{w \in P_v} q_w e^{-\alpha t_w} \right].$$

Furthermore, for each $L \in F_v$ and each $w \in P_v$, $d_L + t_w = t_L$. Thus,

$$z(\text{PSP}) = \sum_{v=1}^M \left[\sum_{L \in F_v} q_L e^{-\alpha t_L} + \sum_{w \in P_v} q_w e^{-\alpha t_w} \right].$$

Since $\{P_1, \dots, P_M, F_1, \dots, F_M\} = \{1, \dots, N\}$, then,

$$z(\text{PSP}) = \sum_{i=1}^N [q_i e^{-\alpha t_i}] = z(P').$$

This completes the proof. $\square$

References

- Bartusch, M., R. H. Mohring, and F. J. Radermacher, "Scheduling Project Networks with Resource Constraints and Time Windows," *Ann. Oper. Res.*, 16 (1988), 201–240.
- Baroum, S. M. and J. H. Patterson, "A Comparative Evaluation of Cash Flow Weight Heuristics for Maximizing the Net Present Value of a Project," Working Paper, Indiana University, Bloomington, IN, 1993.
- Bell, C. E. and K. Park, "Solving Resource-Constrained Project Scheduling Problems by A^* Search," *Naval Res. Logistics*, 37, 1 (1990), 61–84.
- Bey, R. P., R. H. Doersch, and J. H. Patterson, "The Net Present Value Criterion: Its Impact On Project Scheduling," *Project Management Quarterly*, 12, 2 (1981), 35–45.
- Christofides, N., R. Alvarez-Valdez, and J. M. Tamarit, "Project Scheduling with Resource Constraints: A Branch and Bound Approach," *European J. Oper. Res.*, 29, 3 (1987), 262–273.
- Davis, E. W., "Project Scheduling Under Resource Constraints Historical Review and Categorization of Procedures," *AIIE Trans.*, 7, 4 (1973), 297–313.
- and G. E. Heidorn, "An Algorithm for Optimal Project Scheduling Under Multiple Resource Constraints," *Management Sci.*, 17, 12 (1971), B803–16.
- and J. H. Patterson, "A Comparison of Heuristic and Optimum Solutions in Resource Constrained Project Scheduling," *Management Sci.*, 21, 8 (1975), 944–955.
- Demeulemeester, E. and W. Herroelen, "A Branch-and-Bound Procedure for the Multiple Constrained-Resource Project Scheduling Problem," Working Paper, Department of Applied Economic Sciences, Katholieke Universiteit of Leuven, Leuven, Belgium, 1991.
- , "A Branch-and-Bound Procedure for the Multiple Constrained-Resource Project Scheduling Problem," *Management Sci.*, 38, 12 (1992), 1803–1818.
- Doersch, R. H. and J. H. Patterson, "Scheduling a Project to Maximize Its Present Value: A Zero-One Programming Approach," *Management Sci.*, 23, 8 (1977), 882–889.
- Elmaghraby, S. E. and W. S. Herroelen, "The Scheduling of Activities to Maximize the Net Present Value of Projects," *European J. Oper. Res.*, 49, 1 (1990), 35–49.
- Grinold, R. C., "The Payment Scheduling Problem," *Naval Res. Logistics Quarterly*, 19, 1 (1972), 123–136.
- Icmeli, O. and S. S. Erenguc, "A Tabu Search Procedure for Resource Constrained Project Scheduling Problems with Discounted Cash Flows," *Computers and Oper. Res.*, 21, 8 (1994), 841–853.
- , —, and J. C. Zappe, "Project Scheduling Problems: A Survey," *International J. Oper. and Production Management*, 13, 11 (1993), 80–91.
- Kolisch, R., A. Sprecher, and A. Drexl, "Characterization and Generation of a General Class of Resource Constrained Project Scheduling Problems: Easy and Hard Instances," Working Paper, Christian-Albrechts Universitat zu Kiel, Kiel, Germany, 1992.
- Padman, R. and D. E. Smith-Daniels, "Early-Tardy Cost Trade-offs in Resource Constrained Projects with Cash Flows: An Optimization Guided Heuristic Approach," *European J. Oper. Res.*, 64 (1993), 295–311.
- Patterson, J. H., "A Comparison of Exact Approaches for Solving the Multiple Constrained Resource, Project Scheduling Problem," *Management Sci.*, 30, 7 (1984), 854–867.
- Russell, A. H., "Cash Flows in Networks," *Management Sci.*, 16, 5 (1970), 357–373.
- Russell, R. A., "A Comparison of Heuristics for Scheduling Projects with Cash Flows and Resource Restrictions," *Management Sci.*, 32, 10 (1986), 1291–1300.
- Sepil, C., "Dual Simplex Algorithms on Network Flow Problems and Extensions," Unpublished Ph.D. Thesis, University of Florida, 1987.
- Smith-Daniels, D. E. and N. J. Aquilano, "Using a Late-Start Resource-Constrained Project Schedule to Improve Project Net Present Value," *Decision Sci.*, 18, 4 (1987), 617–630.
- Talbot, F. B. and J. H. Patterson, "An Efficient Integer Programming Algorithm with Network Cuts for Solving Resource Constrained Scheduling Problems," *Management Sci.*, 24, 11 (1978), 1197–1174.
- Yang, K. K., F. B. Talbot, and J. H. Patterson, "Scheduling a Project to Maximize its Net Present Value: An Integer Programming Approach," *European J. Oper. Res.*, 64, 2 (1992), 188–198.

Accepted by Thomas M. Lieblich; received June 1994. This paper has been with the authors 5 months for 1 revision.